Back to the spec—its setup will need to create a `Cart` to use in our tests as well as fetch two products from fixtures. By default, fixtures aren't available in specs, but you can call `fixtures` to make them available. Here's what the setup looks like:

rails7/depot_xa/spec/models/cart_spec.rb

```
require 'rails_helper'

RSpec.describe Cart, type: :model do

» fixtures :products
» subject(:cart) { Cart.new }
»
» let(:book_one) { products(:ruby) }
» let(:book_two) { products(:two) }
```

This definitely doesn't look like our original test! The call to `subject` declares the variable `cart`, which you'll use in the tests later. The calls to `let` declare other variables that can be used in the tests. The reason for two methods that seemingly do the same thing is an RSpec convention. The object that's the focus of the test is declared with `subject`. Ancillary data needed for the test is declared with `let`.

The tests themselves will also look different from their equivalents in a standard Rails test. For one thing, they aren't called tests but rather *examples*. Also, it's customary for each example to make only one assertion. The existing test of adding different products makes two assertions, so in the spec, that means two examples.

Assertions look different in RSpec as well:

```
expect(actual_value).to eq(expected_value)
```

Applying this to the two assertions around adding distinct items, we have two examples (we'll show you where this code goes in a moment):

```
it "has two line items" do
  expect(cart.line_items.size).to eq(2)
end
```